

ELO329 - Diseño y programación orientada a objetos

Ayudantía 10

Departamento de Electrónica

Ingeniería Civil Telemática
Byron Agurto, Ignacio González.



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

LIDER EN
INGENIERIA, CIENCIA
Y TECNOLOGIA

1 Resumen de la materia.

2 Contexto

3 Actividad 1

4 Actividad 2

5 Actividad 3

6 Cierre



Esta semana dejaremos de utilizar Java para empezar a trabajar con C++, donde aplicamos la misma teoría vista en Java, solo que con la diferencia de que C++ incluye el manejo de memoria dinámica, herencia múltiple y algunas cosas más. Recordando algunos de los conceptos claves:

- **Clases:** Es análogo a las clases de Java, donde llevamos la abstracción a un nuevo nivel definiendo las clases con sus niveles de acceso, métodos, atributos, constructores y destructores. Donde el funcionamiento de todo lo anterior, salvo los destructores, son análogos a Java.
- **Destructores:** A diferencia de Java, que la JVM se encargaba de liberar la memoria cuando detectaba que no se iba a seguir utilizando, C++ al permitir el manejo de memoria dinámica, no implementa por defecto los destructores y la liberación de memoria, por lo que es necesario implementarlo. Este se llama igual que la clase, solo que se le antepone el carácter " ".
- **Memoria dinámica:** Es la capacidad que le da el lenguaje al programador de utilizar la memoria del **heap** a la manera que la estime conveniente. Esto lo hace haciendo uso de los punteros (apuntan a una dirección de memoria) y las instrucciones **new** y **delete**.



- **Friend:** Es un nuevo calificador que es introducido por C++, y permite que una función sea "*amiga*" de una clase, y esta tenga acceso a todos los métodos y atributos de la clase amiga, se puede aplicar a las clases, donde en este caso los métodos de la clase tendrán acceso a los atributos de la otra, sin embargo hay que considerar que la amistad no es mutua y no es heredable.
- **Sobrecarga de operadores:** Otra de las novedades de C++ es que permite sobrecargar los operadores, es decir, es posible redefinir algunas operaciones para poder operar entre instancias de una clase. Por ejemplo, se podría sobrecargar el operador de la suma dentro de una clase.
- **Herencia:** La herencia en C++ funciona similar a como lo es en Java, salvo que C++ permite que una clase herede de múltiples clases.



- **Métodos virtuales:** A diferencia de Java, cuando se sobrescribe un método en una clase hija, por omisión llamará al método de la clase del tipo de puntero, no al definido en la clase apuntada, lo cual es una diferencia con Java. Si buscamos un comportamiento similar al que había en Java debemos definir el método en la clase padre como *virtual*, que hace que utilice el método definido en la clase apuntada. Existe un tipo de método virtual especial que se les conoce como métodos virtuales puros que son el equivalente a los métodos abstractos en Java, donde se define el método, pero no se implementa.



- **Framework Qt:** Es un framework de desarrollo de aplicaciones muy útil para para desarrollar interfaces gráficas. También introduce mejoras a las bibliotecas nativas de C++, como ocurre con la clase **QString**, la cual viene a mejorar a la clase **String**.
- **Módulos Qt:** Existen dos tipos de módulos en Qt, los **Core-Essential**, que son las que dan forma al framework, aquí se incluyen módulos como **QtCore**, **QtGui** y **QtWidgets**. Por otro lado existen los módulos **add-ons** y aquí se incluyen bibliotecas como **Qt3D**, **QtBluetooth**, **QtSensors**.
- **QtCreator:** Es el IDE que les recomendamos utilizar para trabajar con Qt, ya que si bien muchos IDE permiten una integración, QtCreator es un IDE desarrollado por Qt para facilitar el uso de interfaces gráficas.



- **Signals y Slots:** Permiten el manejo de eventos entre objetos de Qt. (**QObject**)
- **Signals:** Es emitida por un objeto para señalar la ocurrencia de un evento.
- **Slot:** Es un método que es llamado en respuesta a una señal, y se encarga de capturar el evento.



Contexto parte 1

Dawn vió los progresos que llevan hasta el momento con el simulador y ya está notando su utilidad, aunque aún no esté completo, por lo que se está empezando a emocionar con las posibilidades.



Contexto parte 2

En el momento que Piplup se enteró de sus avances se alegró mucho, ya que cada vez falta menos para que pueda dejar de probar las estrategias de Dawn para que Piplup derrote a Pachirisu.



Actividad 1: Interfaz del simulador.

En esta primera etapa deberán agregar una etiqueta a la interfaz que implementaron durante la ayudantía pasada, la cual se utilizará para notificar los movimientos que realice cada pokemon.

Además, deberá crear los movimientos que tendrá Pachirisu a la lista de movimientos ya existentes.

A continuación se muestra un ejemplo de la interfaz que tiene que desarrollar y los movimientos de Pachirisu.



Figure: Ejemplo del simulador (Generado con ChatGPT).

Lista de movimientos de Pachirisu

Nota: La curación aumenta la salud actual en 15 puntos.

Nombre Movimiento	Tipo elemental	Categoría
Chispa	Eléctrico	Ataque
Carga	Eléctrico	Ventaja
Deseo	Normal	Curación
Chispazo	Eléctrico	Ataque

Table: Movimientos de Piplup



Actividad 2: Implementación del sistema de turnos.

- Deberá implementar el sistema de combates por turnos, donde el movimiento que utilice Pachirisu sea decidido al azar.

Nota: Para elegir el movimiento puede apoyarse de las librerías random de C++ o la librería de Qt QRandomGenerator.



Actividad 3: Definición del estado del Pokemon.

Para esta actividad deberá actualizar la barra de vida de cada uno de los pokememon al finalizar cada turno, asegurándose de que no baje de 0, y en caso de que la barra de vida llegue a 0, la etiqueta que mostraba los movimientos realizados muestre un mensaje de la forma:

- Fin del combate!!! <Nombre pokemon> ha quedado incapacitado.

Nota: Cuando finalice el combate, el simulador se detiene y aunque se pulse algún botón, no se aplicará el efecto, ni se modificará la etiqueta.



¡¡¡Felicitaciones!!!

